

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 753 — Computer Vision

Recovering the scene from the image — the inverse of everything CSCE 647 did forward

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 7 (with CSCE 625 and CSCE 642)

Prerequisites: CSCE 641 Computer Graphics and CSCE 647 Image Synthesis for the forward model; CSCE 636 Deep Learning for Modules 08–11; linear algebra, thoroughly

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A working geometric reconstruction of a real scene you captured yourself, with calibration, reprojection error, and an honest account of where it fails — plus a learned-vision component evaluated under distribution shift

Course description

CSCE 647 computed an image from a scene. This course computes a scene from images. That is the entire framing, and it explains both why the subject is hard and why the graphics courses were the right preparation: **you cannot invert a process you do not understand forward.**

The inversion is ill-posed. Infinitely many scenes produce any given image — a small near object and a large far one are indistinguishable, a dark surface in bright light matches a bright surface in dim light. **Module 01 establishes this properly**, because every method in the course is a strategy for constraining the ambiguity: more views, known geometry, motion over time, or a learned prior over what scenes are plausible.

The course has two halves that are genuinely different subjects. Modules 02 through 07 are geometric vision — projective geometry, calibration, correspondence, triangulation, bundle adjustment — which is well-conditioned, provably correct where its assumptions hold, and the basis of every working camera-tracking system. **Modules 08 through 11 are learned vision**, which handles the semantic questions geometry cannot touch and offers no guarantees at all.

They meet at Module 11, where neural scene representations solve the reconstruction problem by optimising a renderer against photographs — **which is CSCE 647's forward model used as a loss function**, and is the clearest demonstration in the program of why the forward and inverse subjects belong together.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why vision is ill-posed and classify the strategies for constraining it.
- 2 Calibrate a camera and state what each intrinsic parameter means.
- 3 Detect, describe, and match features robustly, including RANSAC.
- 4 Derive the epipolar constraint and recover relative pose from two views.
- 5 Run structure from motion and diagnose drift and degeneracy.
- 6 Compute dense depth from stereo and explain where it fails.
- 7 Estimate and track motion, and state the brightness-constancy assumption.
- 8 Train and evaluate detection and segmentation models honestly.
- 9 Explain vision transformers and self-supervised pretraining.
- 10 Explain neural radiance fields and Gaussian splatting as inverse rendering.
- 11 Evaluate a vision system under distribution shift and state what it can be trusted to do.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	The Inverse Problem	Why can an image not simply be read backwards into a scene?
2	Cameras and Calibration	How does a point in the world become a pixel, and how do you measure the mapping?
3	Features and Correspondence	How do you decide that this pixel and that pixel are the same world point?
4	Two-View Geometry	Given two images of a scene, what can be recovered exactly?
5	Structure from Motion	How do you reconstruct a scene from a hundred photographs?
6	Stereo and Dense Depth	How do you get depth everywhere, not just at features?
7	Motion and Tracking	How do you follow a point, or a whole image, from frame to frame?
8	Learned Representations	Why does a model trained on one task help on another?
9	Detection and Segmentation	How do you find and delineate objects, and how do you know whether you did?
10	Transformers and Self-Supervision	What changed when vision adopted the transformer?

Wk	Module	Core question it answers
11	Neural Scene Representations	What if you reconstructed a scene by optimising a renderer against photographs?
12	Evaluation and Failure	Your model scores 94%. On what, and where will it break?
13	Vision in Systems	What changes when the system has to run, not just score?

Projects

Project 1 — Reconstruct a real scene (after Module 7)

Capture a scene yourself, calibrate, and reconstruct its geometry from your own images with your own code.

Requirements

- **Your own calibration** from your own target images, with the intrinsics reported and the reprojection error stated in pixels.
- **Feature detection, matching, and RANSAC** implemented or configured, with the inlier fraction reported.
- **Relative pose from two views**, triangulated to a sparse point cloud.
- **Bundle adjustment** over at least ten views, with the error before and after.
- **A comparison against COLMAP** on the same images.
- **A deliberate failure case**: a textureless surface, a pure rotation, or a repeated pattern, with the symptom diagnosed.

Done means

- **Sub-pixel reprojection error after bundle adjustment**, reported as a number rather than described.
- **The point cloud visibly matches the scene** — rendered from a novel viewpoint and compared against a photograph from roughly that viewpoint.
- **The COLMAP comparison honest**, including where it beats you and by how much.
- **The failure case explained in terms of the geometry**, not described as 'it did not work'.

Project 2 — A learned component, evaluated honestly (after Module 12)

Train or fine-tune a vision model for a task you care about, and establish what it can actually be trusted to do.

Requirements

- **A task with a real use** and a held-out test set collected separately from the training data.
- **A trivial baseline** — CSCE 633's discipline, unchanged.
- **Detection or segmentation metrics computed correctly**, with the IoU threshold stated.
- **An evaluation under distribution shift**: different lighting, camera, or location than training.
- **A failure gallery** of at least twenty cases, grouped by cause.
- **Latency measured** on the hardware it would actually run on.

Done means

- **The shifted-distribution number reported as prominently as the test number**, and it will be worse.

- **The failure modes named and grouped**, with a stated guess at the cause of each group.
- **A statement of what the system can be trusted to do**, scoped to conditions — which is the deliverable.
- **Latency on real hardware**, not on a cloud GPU, if it would deploy on a device.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Szeliski — Computer Vision: Algorithms and Applications, 2nd edition (free PDF)	The primary source, free in full from the author. The geometric chapters are the reference for Modules 02 through 07 and the second edition covers the learned material of Modules 08–10.
Hartley & Zisserman — Multiple View Geometry	The definitive treatment of Modules 04 and 05. Dense and worth the effort; the sample chapters are free and the errata page is useful. Library copy for the rest.
Michigan EECS 498/598 — Deep Learning for Computer Vision (free lectures and assignments)	The best free course for Modules 08 through 10. Full video lectures, slides, and assignments, taught at the right level for someone who has already done CSCE 636.
Mildenhall et al. — NeRF; and Kerbl et al. — 3D Gaussian Splatting (both free)	The two papers Module 11 is built on. Both are readable and both have reference implementations, and the contrast between them is instructive.
OpenCV documentation and tutorials (free)	The working toolkit for Modules 02–07. The calibration and findEssentialMat tutorials are worth following exactly once before writing your own.

Resource	Role
COLMAP documentation (free, open source)	A production structure-from-motion pipeline you can read. Module 05's reference point, and what your own implementation should be compared against.

Tooling

- **A camera you control.** A phone is adequate if you can lock the focus and exposure; **autofocus between frames changes the intrinsics and silently breaks calibration**, which is a lesson best learned once and deliberately.
- **Python with OpenCV, NumPy, and PyTorch.** OpenCV for the geometric pipeline, PyTorch for Modules 08–11.
- **A printed calibration target** — a checkerboard on stiff card, measured. **A target printed at the wrong scale is the most common cause of a calibration that looks fine and is wrong.**
- **COLMAP installed**, as the reference against which your own structure from motion is measured.
- **A GPU for Modules 08–11.** The 4 GB budget of CSCE 636 still applies — NeRF training at reduced resolution and Gaussian splatting on small scenes both fit, with care.
- **A scene you can recapture.** Every geometric failure in this course is diagnosed by capturing again with one thing changed, and a dataset you cannot extend cannot be debugged.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.